

Build a timeserie infrastructure - Take home lab

- [Introduction](#)
- [Duration](#)
- [Target](#)
- [Limitation](#)
- [Evaluation](#)
 - [Bonuses - Level 1](#)
 - [Bonuses - Level 2](#)

Introduction

This test aims to provide insight about how the interviewee approaches a technical problem. We are trying to assess how good an interviewee is and not how bad.

For this reason the choice of the OS (Debian, Ubuntu, Centos, Fedora ...) and the TSDB ([influxdb](#), [prometheus](#), [cortex](#), [thanos](#), [m3db](#), [timescaledb](#), [victoria metrics](#) ...) will be made by the interviewee.



The list of TSDB or OS are not exhaustives

Duration

The interviewee will be given one week to complete this take home lab. However you should not spend more than 5 hours on this matter.

Target

The interviewee will be given access to an [openstack account](#) where he/she will be able to spawn virtual machines. The goal is to build a **scalable** (both RW and RO) and **highly available** time-series databases.

Having an infrastructure up and running does not mean success. Not having an infrastructure up and running does not mean failure.



You can find useful information to start with openstack in our documentation: <https://docs.ovh.com/us/en/public-cloud/>

Limitation



Cloud resources must be used **only** for the technical test.

Resources are limited and usage will be monitored.

You have at your disposal:

- 10 * s1-2
- 5 * s1-8
- 6 * b2-7
- 1 * b2-30
- 6 * c2-7
- 1 * c2-30
- 3 * r2-15
- 1 * r2-60

Use those resources wisely, you can have more information here: <https://www.ovhcloud.com/fr/public-cloud/prices/>

The interviewee will be allowed to used **every resources at its disposal** (meaning internet) as long as **all sources** (links to documentations and/or procedures) **are provided** and the interviewee is able to **explain/justify it**.

Evaluation

- All steps needed for building the cluster **will be documented during the test** by the interviewee, this will be used in post-test debrief

- Given your code and documentation anyone must be able to reach the same result (think like you are open sourcing it)
- Each and every **technical choice must be justified** by the interviewee
- The cluster must **tolerate the loss of one or more servers** (which can be the primary or not)
 - Think chaos monkey!
- The interviewee provide a detailed analysis of his/her lab with
 - How to bench the infrastructure (how many data-points can we write/read per seconds?)
 - All outages possible with their implication and how to fix them
 - Bottlenecks which may arise and how to address them

In order to allow us to assess your infrastructure please add the following sshkey on every host (don't forget to allow incoming traffic)

```
from="51.178.230.230,51.83.57.141" ecdsa-sha2-nistp521
AAAAE2VjZHNhLXNoYTItbmlzdHA1MjEAAAABmlzdHA1MjEAAACFBADJw7M15uB0WU0ByRSVDHqah7Xoat+KE80MQChU3zU5BS5ekdpMSreA0+BB
4VaXFDeAG5y/ZmaNBM7e6ixs0w5DmAE2H4Nlk5kVbvrQdbAZVG3mn+85dYzZBqISmve/hl0rn2NiHN0b8bHQveJw7ys1o6t
/Rkg0rcNMSMyDaiyGiH9iuA== observability-int@zpub:1641904424
```

Bonuses - Level 1

- Bench your infrastructure
- Include a service discovery
- Deploy with infrastructure as code & automation tools
- Provide dashboards to exploit the newly created infrastructure
- Collect metrics about your metrics infrastructure

Bonuses - Level 2

One week is more than enough for you to complete this test and the first bonuses level?

Improve as much as you can the infrastructure reliability, maintainability, observability and so on.

Have fun, be creative. Feel free to show us what you got!

Still reading? Do you need idea for the level 2? Check this:

- Test your code (infra-as-code & automation tools usually provide testing features)
- Make sure you are following the best practices of your selected tools
- Increase the security level of the infrastructure
- Reduce deployment time
- Tune the infrastructure to improve its performance
- Collect logs
- Build meaningful dashboards